

Framework / Tool box / Not an AI model.



Date: . . . . .

## LangChain Agents & Chains.

LangChain takes ReAct pattern (Reason + Act + Loop) we coded by hands & turns it into clean, reusable framework for building LLM agents.

• Why use Framework LangChain?

a) Reduce Boilerplate code → Manual agents require writing loops ...

• LangChain automates all that plumbing we focus on logic

b) Standard tool Interface

• without LangChain each function has its own custom format.

• LangChain enforces uniform tool structure so LLM knows how to use it.

c) Observability & debugging

• LangChain has built-in tracing like we can see every step so easier to debug

d) Plug & Play Ecosystem

• Pre-built integrations for tools & APIs

• LangChain → build on top of LLMs.

• Ideas →

- Chains (pre-defined workflow where each step is fixed)
- Agents (decide dynamically which tool to use)

Chains (Fixed recipes)

• Example: Take input → Summarize  
Return ← Translate

• Always follow same path, no matter of output.

• Not flexible → Can't adapt to new query

• Use when we know path

Agents (Adaptive Chains)

If maths question → call calculator.

• Use when path depends on question.



Date: \_\_\_\_\_

## Manual ReAct

- We wrote prompt
- Tools plain python function.
- Adding tools or parsing errors.
- Debugging harder

## LangChain Agent.

- Built-in ReAct prompts & managed Reason → Act → <sup>observ.</sup> loop
- Tools are wrapped in a standard tool format.
- Agent Executor runs loop & handles parsing auto.
- Built-in step tracing & logs - debugging easy.